

Laboratorio de Red Virtualizada con KVM/QEMU y Monitoreo Centralizado con Zabbix

Manual técnico de despliegue, configuración y resolución de problemas

Documentación de laboratorio personal
Analista de Sistemas · Técnico Jr. en Redes Cisco (CCNA 1-2-3) · DevOps
Septiembre 2026

Índice

Introducción

Alcance del documento

1. Arquitectura general del laboratorio

2. Prerrequisitos

Hardware utilizado en este laboratorio

Software

3. Parte 1 — Hipervisor: KVM/QEMU sobre Ubuntu Server

3.1. Verificar soporte de virtualización por hardware

3.2. Instalar KVM, QEMU y herramientas de gestión

3.3. Dar permisos a tu usuario habitual

4. Parte 2 — Creación de las máquinas virtuales

4.1. Descargar las imágenes ISO

4.2. Crear la VM con Virtual Machine Manager (virt-manager)

4.3. Instalación de Debian 12 (resumen de opciones clave)

4.4. Instalación de Fedora Server (resumen de opciones clave)

5. Parte 3 — Acceso remoto: arquitectura Bastion Host

5.1. Puntos clave de conectividad

5.2. Verificación de conectividad interna

5.3. Conexión en dos saltos (método simple)

5.4. Conexión en un solo comando (`-J` , ProxyJump)

6. Parte 4 — Zabbix Agent en las VMs

6.1. Instalación

6.2. Localizar el archivo de configuración correcto

6.3. Parámetros a modificar

6.4. Firewall (solo Fedora)

7. Parte 5 — Zabbix Server dockerizado

7.1. Instalar Docker y Docker Compose

7.2. Liberar puertos si ya existía un Zabbix nativo

7.3. `docker-compose.yml`

7.4. Levantar el stack

7.5. Acceder a la interfaz web

8. Parte 6 — Alta de hosts y templates en Zabbix

8.1. Crear un host

8.2. Asignar el template

8.3. Validar que los datos estén llegando

9. Integración de Zabbix con Google Gemini (diagnóstico de alertas on-demand)

9.1. Configuración de la alerta / script en Zabbix

9.2. Parámetros configurados

9.3. Script funcional (JavaScript, webhook de Zabbix)

9.4. Modo de uso

9.5. Problemas frecuentes

10. Runbook de resolución de problemas

Incidentes reales y su resolución

11. Checklist final de verificación

12. Próximos pasos sugeridos

Introducción

Este documento reconstruye, en formato de manual técnico (runbook), el proceso completo de despliegue de un laboratorio de red virtualizada sobre un servidor físico con **Ubuntu Server**, monitoreado de forma centralizada con **Zabbix**. Está pensado para que otra persona —o el propio autor, en una instalación futura— pueda repetir el proceso de cero, entendiendo el *por qué* de cada decisión y los problemas reales que aparecieron en el camino.

Por motivos de seguridad, **se omitieron o reemplazaron por marcadores genéricos todos los datos que podrían exponer la red del autor** (direcciones IP de la red doméstica, nombres de usuario reales, contraseñas).

Alcance del documento

- 1. Instalación y configuración del hipervisor KVM/QEMU sobre Ubuntu Server.
- 2. Creación de dos máquinas virtuales — Debian 12 y Fedora Server — para simular una LAN de servidores.
- 3. Arquitectura de acceso remoto vía SSH (Bastion Host).
- 4. Gestión básica de usuarios, grupos y permisos en Linux.
- 5. Instalación de Zabbix Agent en ambas VMs.
- 6. Despliegue de Zabbix Server dockerizado en una notebook de administración.
- 7. Alta de hosts, templates y validación del monitoreo en la interfaz web.
- 8. Integración de Zabbix con Google Gemini para diagnóstico de alertas on-demand.
- 9. Runbook de resolución de problemas comunes, basado en incidentes reales del proyecto.

1. Arquitectura general del laboratorio

El laboratorio combina dos equipos físicos y dos máquinas virtuales:

- **Host físico A — Servidor Ubuntu Server:** actúa como hipervisor (KVM/QEMU + libvirt) y como *Bastion Host* (puerta de entrada SSH) hacia las VMs.
- **VM 1 — Debian 12 Server:** instalada dentro del host anterior, representa la familia Debian/Ubuntu (`apt`).
- **VM 2 — Fedora Server:** instalada dentro del mismo host, representa la familia Red Hat (`dnf` , `firewalld` , `cockpit`).
- **Host físico B — Notebook de administración:** corre una VM con **Debian Trixie**, dentro de la cual se despliega el **Zabbix Server** (dockerizado) que centraliza el monitoreo de los tres nodos anteriores.



Decisión clave de red: como las VMs de KVM viven detrás de una red NAT privada que solo existe dentro del host físico A, el Zabbix Server (ubicado en otra red) **no puede iniciar conexión** hacia ellas. Por eso el monitoreo se configura en **modo agente activo** (`Zabbix Agent Active`): son las VMs las que inician la conexión saliente hacia el servidor, algo que el NAT sí permite sin

configuración adicional.

2. Prerrequisitos

Hardware utilizado en este laboratorio

El siguiente es el hardware con el que efectivamente se armó este laboratorio. No es una recomendación de mínimos ni una configuración ideal para producción, pero resultó suficiente para prácticas sencillas de virtualización y monitoreo con Linux:

- CPU con soporte de virtualización por hardware (Intel VT-x / AMD-V).
- 2 núcleos físicos (se asignó 1 vCPU por VM).
- 16 GB de RAM (se asignaron 2 GB por VM, dejando margen para el sistema anfitrión).
- Al menos 40-50 GB de disco libre para las dos imágenes ISO y los discos virtuales.

Software

- Ubuntu Server (con `sudo add-apt-repository universe` habilitado).
- Acceso a Internet para descargar paquetes e imágenes ISO.
- Cliente SSH (PowerShell/OpenSSH en Windows, o terminal en Linux/macOS).
- Una segunda máquina (notebook de administración) con VirtualBox y una VM Debian Trixie, donde correrá el Zabbix Server.

3. Parte 1 — Hipervisor: KVM/QEMU sobre Ubuntu Server

3.1. Verificar soporte de virtualización por hardware

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

Un valor mayor a `0` confirma que la virtualización por hardware está habilitada. Si devuelve `0`, hay que activarla desde la BIOS/UEFI.

3.2. Instalar KVM, QEMU y herramientas de gestión

```
sudo apt update
sudo apt install -y qemu-system-x86 qemu-kvm libvirt-daemon-system \
    libvirt-clients bridge-utils virtinst virt-manager
```

Verificación del demonio de virtualización:

```
systemctl status libvirtd
```

Debe figurar como `active (running)`.

Nota de troubleshooting: si `apt` no encuentra los paquetes (`Unable to locate package qemu-kvm`), verificá dos cosas antes de nada: 1. Que el repositorio `universe` esté habilitado: `sudo add-apt-repository universe && sudo apt update`. 2. Que no haya quedado agregada la arquitectura secundaria `i386`, que a veces rompe la resolución de paquetes de virtualización de 64 bits: `sudo dpkg --remove-architecture i386 && sudo apt update`.

3.3. Dar permisos a tu usuario habitual

Para administrar VMs sin usar `sudo` en cada comando:

```
sudo usermod -aG libvirt,kvm <tu_usuario>
groups <tu_usuario> # confirmar que aparezcan libvirt y kvm
```

- **-a** (append) es fundamental: sin ella, **usermod -G** **reemplaza** todos los grupos secundarios del usuario en lugar de sumarle uno nuevo (riesgo real de sacarlo por accidente del grupo **sudo**).
- Los cambios de grupo se aplican recién en la próxima sesión: cerrá y volvé a abrir la sesión SSH, o ejecutá **newgrp libvirt**.

4. Parte 2 — Creación de las máquinas virtuales

4.1. Descargar las imágenes ISO

Guardar siempre las ISO en el directorio estándar de **libvirt** para evitar problemas de permisos:

```
# Debian 12 (netinst, ~600 MB)
sudo wget -O /var/lib/libvirt/images/debian-12-netinst.iso \
https://get.debian.org/images/release/current/amd64/iso-cd/debian-netinst.iso
```

Fedora Server no tiene un enlace de descarga directa tan estable; se recomienda bajarlo manualmente desde <https://fedoraproject.org/server/download/> (opción **Server Netinstall**, ~1-1.2 GB) y luego moverlo:

```
sudo mv ~/Descargas/Fedora-Server-*.iso /var/lib/libvirt/images/
ls -lh /var/lib/libvirt/images/ # confirmar que ambas ISO estén presentes
```

Troubleshooting: en Linux las rutas son sensibles a mayúsculas/minúsculas — **/home** existe, **/Home** no. Si tu sistema está en español, la carpeta de descargas se llama **Descargas**, no **Downloads**. Usá la tecla **Tab** para autocompletar rutas y evitar errores de tipeo.

Si después de mover una ISO no aparece en **virt-manager**, hacé clic en el ícono de refrescar (flecha circular) junto al selector de volúmenes: la interfaz gráfica no detecta automáticamente los cambios hechos por consola.

4.2. Crear la VM con Virtual Machine Manager (virt-manager)

Para cada máquina (Debian y Fedora) el asistente sigue el mismo patrón:

Paso	Configuración
Medio de instalación	Local install media (ISO image or CDROM)
ISO	Seleccionar el archivo .iso correspondiente desde el pool default
Memoria	2048 MiB (2 GB)
CPUs	1 vCPU
Disco	15.0 GiB, formato qcow2
Nombre	debian-server / fedora-server
Red	Virtual network 'default': NAT (conecta al bridge virbr0)

4.3. Instalación de Debian 12 (resumen de opciones clave)

- **Hostname:** **debian-server**.
- **Usuario y contraseña:** crear un usuario no privilegiado (no root) y anotar bien las credenciales.
- **Particionado:** *Guiado - utilizar todo el disco* → *Todos los ficheros en una partición* (para un disco de 15 GB dedicado a laboratorio, separar **/home** solo agrega complejidad innecesaria).
- **Selección de programas (tasksel):** desmarcar *Debian desktop environment* y *GNOME*; dejar marcados únicamente **SSH server** y **standard system utilities**. Esto evita instalar un entorno gráfico innecesario y deja la VM con menos de 200 MB de RAM en reposo.
- **GRUB:** instalar en **/dev/vda** (el disco virtual asignado; en KVM los discos usan el prefijo **vd*** por el controlador

paravirtualizado VirtIO, no `sd*`).

Tras el primer arranque, como Debian mínimo **no trae `sudo` instalado por defecto**:

```
su - # entrar como root
apt update && apt install -y sudo
usermod -aG sudo <tu_usuario>
exit
```

Obtener la IP asignada por DHCP dentro del rango NAT de KVM:

```
ip -br a # buscar la interfaz ena/enp... (no virbr0)
```

4.4. Instalación de Fedora Server (resumen de opciones clave)

- **Hostname:** `fedora-server`.
- **Usuario:** crearlo marcando la opción “Hacer que este usuario sea administrador” (lo agrega al grupo `wheel`, equivalente a `sudo` en la familia Red Hat).
- **Particionado:** automático sobre el disco virtual de 15 GB.

Configuraciones post-instalación recomendadas:

```
# Panel de administración web (opcional, puerto 9090)
sudo systemctl enable --now cockpit.socket

# Verificar que el firewall (firewalld) permita SSH
sudo firewall-cmd --list-services
# Debe listar al menos: ssh, dhcpv6-client (y cockpit si se habilitó)
```

Troubleshooting — renombrar un usuario mal escrito: si el usuario quedó con un nombre incorrecto durante la instalación, no hace falta reinstalar. Desde una sesión de **root** (o desde otra terminal virtual, `Ctrl+Alt+F2`, si el usuario tiene procesos activos):

```
kill -u <usuario_viejo> # cortar procesos/sesión activa
usermod -l <usuario_nuevo> -d /home/<usuario_nuevo> -m <usuario_viejo>
groupmod -n <usuario_nuevo> <usuario_viejo>
id <usuario_nuevo> # verificar UID/GID y grupos
```

Si el sistema insiste en que el usuario “está en uso” incluso tras matar sus procesos (por reinicio automático de la sesión de `systemd -user`), cortá la sesión de forma explícita antes de reintentar:

```
loginctl terminate-user <usuario_viejo>
```

Como alternativa más simple, también es válido crear el usuario correcto de cero y borrar el anterior:

```
useradd -m -G wheel <usuario_nuevo>
passwd <usuario_nuevo>
userdel -r -f <usuario_viejo>
```

5. Parte 3 — Acceso remoto: arquitectura Bastion Host

Con las dos VMs operativas, los tres nodos (host físico + 2 VMs) quedan conectados al mismo switch virtual `virbr0`, en el mismo dominio de broadcast (a modo de ejemplo, `10.10.10.0/24`).

5.1. Puntos clave de conectividad

- La red interna del switch virtual es **privada e interna** del host con KVM. Ningún otro equipo de la red doméstica (por ejemplo, una notebook por Wi-Fi) puede alcanzarla directamente: hay que entrar primero por SSH al host físico y desde ahí “saltar” a las VMs.

- El host físico Ubuntu cumple entonces el rol de **Bastion Host / Jump Host**.

5.2. Verificación de conectividad interna

```
ip link show master virbr0      # lista las interfaces vnetX de cada VM
ping -c 3 <IP_VM_DESTINO>      # ping entre VMs, debería dar 0% packet loss
```

5.3. Conexión en dos saltos (método simple)

```
# 1) Desde el cliente, conectar al host físico
ssh <usuario>@<IP_HOST_UBUNTU>

# 2) Ya dentro del host, saltar a la VM
ssh <usuario_vm>@192.168.X.X
```

5.4. Conexión en un solo comando (**-J**, ProxyJump)

```
ssh -J <usuario>@<IP_HOST_UBUNTU> <usuario_vm>@192.168.X.X
```

-J le indica al cliente SSH que use el host físico como intermediario (*jump*) para llegar directamente a la VM, sin quedar logueado en el host intermedio.

Troubleshooting frecuente: un error típico es intentar conectarse desde una notebook externa **directamente** a la IP interna de la VM — el intento queda colgado hasta **Connection timed out**, porque esa red no es enrutable desde afuera del host físico. Siempre hay que apuntar primero a la IP real del host físico en la red local.

6. Parte 4 — Zabbix Agent en las VMs

6.1. Instalación

```
# Debian
sudo apt update && sudo apt install -y zabbix-agent

# Fedora
sudo dnf install -y zabbix-agent
```

6.2. Localizar el archivo de configuración correcto

Este fue uno de los puntos que más tiempo insumió en el proyecto: **la ruta del archivo de configuración no es la misma en todas las distribuciones ni en todos los paquetes**. Antes de editar a ciegas, confirmá qué archivo lee realmente el servicio:

```
systemctl cat zabbix-agent
```

Buscá la línea **ExecStart=**. Si el binario se ejecuta solo con **-f** (sin **-c /ruta/archivo.conf**), está usando la ruta compilada por defecto, que puede ser **/etc/zabbix_agentd.conf** (a veces el caso en Fedora) en lugar de la ruta “esperada” **/etc/zabbix/zabbix_agentd.conf** (la habitual en Debian). Ante la duda, confirmá con:

```
find /etc/ -name "*zabbix*agent*.conf" 2>/dev/null
grep -rn "^Hostname=" /etc/
```

6.3. Parámetros a modificar

En el archivo de configuración correcto, ajustar exactamente estas tres directivas:

```
Server=<IP_ZABBIX_SERVER>
ServerActive=<IP_ZABBIX_SERVER>
Hostname=<Nombre-Exacto-Del-Host>
```

- **Server**: IP desde la que se aceptan chequeos entrantes (pasivos).
- **ServerActive**: IP a la que el agente **envía** sus métricas por su cuenta (chequeos activos). Es la directiva crítica en este esquema, porque las VMs están detrás de NAT y deben iniciar ellas la conexión.
- **Hostname**: debe coincidir **letra por letra, mayúsculas incluidas**, con el nombre del host dado de alta en la interfaz web de Zabbix. Es la causa más común de que un host nunca se vincule.

Troubleshooting: si el agente arranca y en el log dice `unable to connect to [127.0.0.1:10051]`, significa que **ServerActive** quedó con el valor de fábrica (`127.0.0.1`), es decir “el servidor está en esta misma máquina”) y no se editó.

Reiniciar y verificar:

```
sudo systemctl restart zabbix-agent
sudo systemctl enable zabbix-agent
sudo systemctl status zabbix-agent          # debe decir active (running)
sudo journalctl -u zabbix-agent -n 15 --no-pager # confirmar el Hostname real de arranque
```

6.4. Firewall (solo Fedora)

Fedora trae `firewalld` activo por defecto. Si en algún momento se necesitan chequeos pasivos (puerto 10050 entrante):

```
sudo firewall-cmd --add-port=10050/tcp --permanent
sudo firewall-cmd --reload
```

7. Parte 5 — Zabbix Server dockerizado

Se optó por **dockerizar únicamente el Zabbix Server** (base de datos + backend + frontend web), dejando el **agente instalado de forma nativa** en cada nodo monitoreado. Razones:

- El agente necesita medir métricas reales del sistema operativo anfitrión; si se dockerizara, quedaría aislado del hardware/SO real que se quiere observar.
- El servidor, en cambio, se beneficia de la portabilidad y el aislamiento de dependencias que da Docker (no hace falta instalar manualmente PostgreSQL/MySQL, PHP ni un servidor web en el sistema base).

7.1. Instalar Docker y Docker Compose

En Debian Trixie, el paquete se llama `docker-compose` (a secas), no `docker-compose-plugin` (ese es el nombre del repositorio propietario de Docker Inc.):

```
sudo apt update
sudo apt install -y docker.io docker-compose
sudo systemctl enable --now docker
sudo usermod -aG docker $USER && newgrp docker
```

7.2. Liberar puertos si ya existía un Zabbix nativo

Si en esa misma máquina había una instalación tradicional de Zabbix (con Apache), hay que detenerla antes de levantar los contenedores para evitar conflictos de puertos (80/8080, 10051, 5432/3306):

```
sudo systemctl stop zabbix-server zabbix-agent apache2
sudo systemctl disable zabbix-server zabbix-agent apache2
```

7.3. `docker-compose.yml`

```
mkdir -p /opt/zabbix-docker && cd /opt/zabbix-docker
nano docker-compose.yml
```

```
version: '3.8'

services:
  zabbix-db:
    image: postgres:15-alpine
    container_name: zabbix-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: <definir_contraseña_segura>
      POSTGRES_DB: zabbix
    volumes:
      - db_data:/var/lib/postgresql/data

  zabbix-server:
    image: zabbix/zabbix-server-pgsql:latest
    container_name: zabbix-server
    restart: unless-stopped
    ports:
      - "10051:10051"
    environment:
      DB_SERVER_HOST: zabbix-db
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: <definir_contraseña_segura>
      POSTGRES_DB: zabbix
    depends_on:
      - zabbix-db

  zabbix-web:
    image: zabbix/zabbix-web-nginx-pgsql:latest
    container_name: zabbix-web
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      ZBX_SERVER_HOST: zabbix-server
      DB_SERVER_HOST: zabbix-db
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: <definir_contraseña_segura>
      POSTGRES_DB: zabbix
    depends_on:
      - zabbix-server

volumes:
  db_data:
```

Reemplazar `<definir_contraseña_segura>` por una contraseña real (idéntica en las tres apariciones) antes de levantar el stack. No usar contraseñas de ejemplo en un entorno que vaya a exponerse fuera del laboratorio.

7.4. Levantar el stack

```
docker compose up -d      # o "docker-compose up -d" según la versión instalada
docker compose ps         # los 3 contenedores deben figurar como "Up"
```

7.5. Acceder a la interfaz web

```
http://<IP_NOTEBOOK_ADMIN>:8080
```

- Usuario por defecto: `Admin`
- Contraseña por defecto: `zabbix` (cambiarla al primer ingreso)

8. Parte 6 — Alta de hosts y templates en Zabbix

8.1. Crear un host

1. Menú lateral → **Data collection** → **Hosts** → **Create host**.
2. **Host name**: debe ser **idéntico**, carácter por carácter, al valor de `Hostname=` configurado en el agente (por ejemplo `Debian-Server`).
3. **Visible name** (opcional): un nombre descriptivo para mostrar en pantalla.
4. **Host groups**: por ejemplo `Linux servers`.
5. **Interfaces** → **Add** → **Agent**: IP interna de la VM, puerto `10050`.
6. Guardar con **Add**.

8.2. Asignar el template

En versiones recientes de Zabbix, el campo de plantillas no autocompleta texto libre; hay que usar el selector:

1. Dentro de la ficha del host, en **Templates**, hacer clic en **Select**.
2. **Template group** → `Templates/Operating systems`.
3. Marcar **Linux by Zabbix agent active** (no la versión pasiva — ver nota más abajo).
4. **Select** → **Update**.

¿Por qué template activo y no el estándar (pasivo)? Porque el Zabbix Server no tiene ruta directa hacia la red NAT interna de KVM; si se usara el template pasivo, el servidor intentaría iniciar la conexión hacia la VM y fallaría con `Connection timed out`. En el modo activo es la VM la que sale hacia el servidor, y el NAT permite el tráfico saliente sin configuración adicional.

8.3. Validar que los datos estén llegando

El ícono de disponibilidad (**ZBX**) en la lista de hosts **puede quedar en gris** incluso con el monitoreo funcionando correctamente: ese ícono refleja chequeos *pasivos*, y algunas versiones de Zabbix Agent (la rama clásica 6.x) no envían el “heartbeat” de disponibilidad en modo exclusivamente activo. La forma confiable de confirmar que todo funciona es:

Monitoring → **Latest data** → **filtrar por Host** → **Apply**

Si aparecen métricas de CPU, memoria, disco e interfaces de red con marca de tiempo reciente, el host está monitoreado correctamente, más allá del color del ícono.

9. Integración de Zabbix con Google Gemini (diagnóstico de alertas on-demand)

Con el monitoreo ya funcionando, se sumó una capa adicional sobre Zabbix: la posibilidad de pedir, a demanda, un análisis automatizado de causa raíz y comandos de mitigación para cualquier alerta activa, usando la API de **Google Gemini** directamente desde la interfaz web de Zabbix, sin componentes externos (ni Python, ni contenedores adicionales).

La implementación permite a los operadores de monitoreo solicitar ese análisis directamente desde el menú contextual de cualquier evento o problema activo en **Monitoring** → **Problems**.

9.1. Configuración de la alerta / script en Zabbix

- **Ruta**: `Alerts` → `Scripts` → `Create script` (o editar uno existente).
- **Name**: `Posible causa y solución`
- **Scope**: `Manual event action` (permite ejecutarlo a demanda haciendo clic sobre la alerta en *Monitoring* → *Problems*).
- **Menu path**: `Consulta IA`
- **Type**: `Webhook`
- **Timeout**: `30s`

9.2. Parámetros configurados

Name	Value	Descripción
alert_subject	{TRIGGER.NAME}	Macro de Zabbix que envía el nombre del incidente/disparador.
api_key	<TU_API_KEY_DE_GEMINI>	Clave de autenticación generada en Google AI Studio.
model	gemini-3.5-flash-lite	Identificador del modelo activo, parametrizado para poder cambiarlo sin tocar el código del script.

9.3. Script funcional (JavaScript, webhook de Zabbix)

Los *webhooks* de Zabbix se definen en JavaScript (motor Duktape embebido), no en Python. El modelo y la API key se leen como parámetros en lugar de quedar *hardcodeados* en el código:

```
try {
  var params = JSON.parse(value);
  var subject = params.alert_subject || 'Alerta de prueba en Zabbix';
  var apiKey = params.api_key;
  var model = params.model || 'gemini-3.5-flash-lite';

  if (!apiKey || apiKey === '') {
    throw 'Falta el parametro api_key.';
  }

  var promptText = "La alerta: " + subject + " ocurrió en Zabbix.\n" +
    "Sugiere posibles causas y soluciones tecnicas para resolver este problema.\n" +
    "Respuesta concisa (maximo 10 lineas) con causas probables y 3 comandos Linux de triage/mitigacion.";

  var request = new HttpRequest();
  request.addHeader('Content-Type: application/json');
  request.addHeader('x-goog-api-key: ' + apiKey.trim());

  var payload = {
    contents: [
      { parts: [ { text: promptText } ] }
    ],
    generationConfig: {
      temperature: 0.2,
      maxOutputTokens: 400
    }
  };

  var url = 'https://generativelanguage.googleapis.com/v1beta/models/' + model.trim() + ':generateContent';
  var response = request.post(url, JSON.stringify(payload));

  if (request.getStatus() !== 200) {
    throw 'Error HTTP ' + request.getStatus() + ': ' + response;
  }

  var parsed = JSON.parse(response);
  if (!parsed.candidates || parsed.candidates.length === 0) {
    throw 'Respuesta vacia de Gemini: ' + response;
  }

  return parsed.candidates[0].content.parts[0].text.trim();
} catch (error) {
  Zabbix.log(3, '[Gemini Webhook] Error: ' + error);
  throw 'Fallo en análisis IA: ' + error;
}
```

9.4. Modo de uso

1. Ir a **Monitoring → Problems**.
2. Hacer clic sobre cualquier problema, o en su columna **Actions**.
3. Seleccionar **Consulta IA → Posible causa y solución**.

4. La ventana emergente muestra el diagnóstico sintético junto con los comandos de triage listos para usar en la terminal del servidor afectado.

9.5. Problemas frecuentes

- **Modelo no disponible o dado de baja:** los modelos de Gemini se van reemplazando con el tiempo — en este proyecto, una versión que funcionaba dejó de estar disponible de un día para el otro. Por eso el script toma el modelo desde el parámetro `model` en lugar de tenerlo fijo en el código: alcanza con actualizar ese parámetro en Zabbix a una versión vigente, sin editar el webhook.
- **Error 400 - API key not valid:** el error no siempre es por el modelo — a veces la clave en sí es inválida o quedó mal copiada. Antes de cargar una API key nueva en Zabbix, conviene validarla directamente contra la API:

```
curl "https://generativelanguage.googleapis.com/v1beta/models?key=<TU_API_KEY>"
```

Una clave inválida devuelve un JSON de error similar a este:

```
{
  "error": {
    "code": 400,
    "message": "API key not valid. Please pass a valid API key.",
    "status": "INVALID_ARGUMENT"
  }
}
```

Si se generaron varias claves de prueba, repetir el comando con cada una hasta que la respuesta sea `HTTP 200` con el listado de modelos disponibles confirma cuál es la clave realmente activa para cargar en el parámetro `api_key` del script.

10. Runbook de resolución de problemas

Tabla de comandos de diagnóstico usados durante el proyecto para destrabar la comunicación agente → servidor:

Comando	Para qué sirve
<code>sudo dnf install -y curl / sudo apt install -y curl</code>	Instalar <code>curl</code> si no está disponible, para probar conectividad TCP.
<code>curl -v telnet://<IP_ZABBIX_SERVER>:10051</code>	Verifica el <i>handshake</i> TCP contra el puerto del Zabbix Server (descarta bloqueos de firewall/enrutamiento). Cortar con <code>Ctrl+C</code> tras un par de segundos.
<code>timeout 3 bash -c "</dev/tcp/<IP_ZABBIX_SERVER>/10051" && echo ABIERTO \ echo BLOQUEADO</code>	Misma verificación, sin depender de <code>curl</code> (usa sockets nativos de Bash).
<code>ping -c 3 <IP_DESTINO></code>	Confirma conectividad de red básica (capa 3) antes de sospechar de puertos o del propio Zabbix.
<code>systemctl cat zabbix-agent</code>	Muestra la definición real del servicio <code>systemd</code> , incluida la ruta del <code>.conf</code> que efectivamente usa el binario.
<code>grep -rn "^Hostname=" /etc/</code>	Ubica, sin ambigüedad, en qué archivo está definido el <code>Hostname</code> activo.
<code>grep -E "^(Server ServerActive Hostname HostnameItem)=\"" <ruta.conf></code>	Extrae los parámetros efectivos, ignorando comentarios.
<code>ls -la /etc/zabbix/zabbix_agentd.conf.d/</code>	Descarta que un archivo de inclusión (<code>Include=</code>) esté sobrescribiendo la configuración principal.
<code>sudo systemctl restart zabbix-agent</code>	Aplica en memoria los cambios guardados en el archivo de configuración.
<code>sudo journalctl -u zabbix-agent -n 15 --no-pager</code>	Confirma con qué <code>Hostname</code> arrancó realmente el agente (debe coincidir con el de la web).
<code>docker exec -it zabbix-server zabbix_server -R config_cache_reload</code>	Fuerza al servidor a releer inmediatamente la lista de hosts, sin esperar el ciclo automático (~2 min).
<code>docker logs --tail 30 zabbix-server \ grep -iE "<Nombre-Host> active check"</code>	Revisa, del lado del servidor, si está recibiendo (o rechazando) el tráfico de un host puntual.

Incidentes reales y su resolución

A. Ruta de configuración distinta entre Debian y Fedora. Al abrir `/etc/zabbix/zabbix_agentd.conf` en Fedora, el archivo aparecía vacío. El diagnóstico con `systemctl cat zabbix-agent` reveló que el binario arrancaba únicamente con `-f` (sin `-c`), por lo que tomaba la ruta compilada por defecto: `/etc/zabbix_agentd.conf`, en la raíz de `/etc/`. **Solución:** editar el archivo que el servicio realmente carga, confirmado con el comando anterior — no asumir que la ruta es igual en todas las distribuciones.

B. El servidor rechaza las métricas por Hostname distinto. El log del agente mostraba `Starting Zabbix Agent [Zabbix server]` (el valor de fábrica) en lugar del nombre configurado. Los chequeos activos exigen coincidencia exacta entre `Hostname=` del agente y el *Host name* dado de alta en la web. **Solución:** fijar explícitamente `Hostname=<Nombre-Exacto>` en el archivo correcto, reiniciar el agente y confirmar en el log de arranque (`journalctl`) que el nombre coincide letra por letra.

C. Ícono de disponibilidad en gris con datos fluyendo con normalidad. Ocurrió en el nodo con la versión clásica de Zabbix Agent (rama 6.x) usando el template exclusivamente activo. **Diagnóstico:** el comportamiento es esperado — el ícono ZBX depende de chequeos pasivos o de un *heartbeat* de disponibilidad que solo incorporan versiones más nuevas del agente. **Validación:** se confirmó el flujo real de datos en *Monitoring* → *Latest data*, no por el color del ícono.

D. Conexión rechazada a 127.0.0.1:10051. El agente intentaba conectarse a sí mismo porque `ServerActive` conservaba el valor de fábrica. **Solución:** reemplazarlo por la IP real del Zabbix Server.

E. Instalación de paquetes con nombre mal escrito. Errores como `Unable to locate package qemu-system-x86` correspondían a errores de tipo (`qemu` en vez de `qemu`, `kmv` en vez de `kvm`). Antes de suponer un problema de repositorios, conviene releer el nombre exacto del paquete carácter por carácter.

11. Checklist final de verificación

- ☐ `systemctl status libvirtd` → `active (running)` en el host físico.
- ☐ `virsh list --all` (o `virt-manager`) muestra las dos VMs (`debian-server`, `fedora-server`) corriendo.
- ☐ Se puede hacer SSH al host físico desde la notebook cliente.

- ☐ Se puede hacer SSH desde el host físico a cada VM.
 - ☐ `ping` cruzado entre las dos VMs con 0% de pérdida de paquetes.
 - ☐ `systemctl status zabbix-agent` → `active (running)` en ambas VMs.
 - ☐ `Server`, `ServerActive` y `Hostname` correctamente configurados y verificados con `journalctl` en ambos agentes.
 - ☐ `docker compose ps` en la notebook de administración muestra `zabbix-db`, `zabbix-server` y `zabbix-web` como `Up`.
 - ☐ Acceso al panel web de Zabbix (`http://<IP_NOTEBOOK_ADMIN>:8080`) y contraseña de `Admin` cambiada.
 - ☐ Ambos hosts dados de alta en **Data collection → Hosts**, con el template **Linux by Zabbix agent active** asignado.
 - ☐ **Monitoring → Latest data** muestra métricas recientes (CPU, memoria, disco, red) para los dos hosts.
-

12. Próximos pasos sugeridos

Estas líneas de trabajo quedaron identificadas como continuación natural del laboratorio, pero no forman parte de esta documentación:

- **Autenticación SSH por llave pública/privada**, eliminando el uso de contraseñas interactivas para el acceso a las VMs.
- **Automatización con Bash** de tareas de mantenimiento recurrentes, como respaldos periódicos de archivos de configuración críticos.
- Evaluar la actualización del agente en el nodo Debian a **Zabbix Agent 2**, que sí reporta *heartbeat* de disponibilidad en modo activo (evitaría el ícono gris descrito en el incidente C).